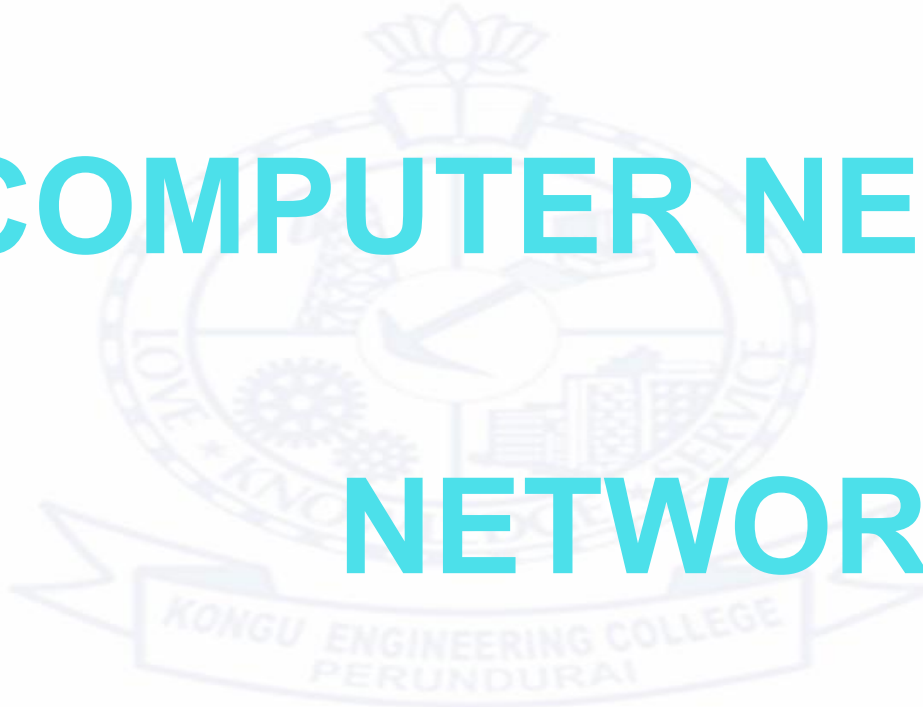


The logo of Kongu Engineering College is a circular emblem. It features a central gear with a lotus flower on top. The text 'LOVE KNOWLEDGE SERVICE' is written around the inner circle. Below the emblem, a banner reads 'KONGU ENGINEERING COLLEGE PERUNDURAI'.

COMPUTER NETWORKS

UNIT-3

NETWORK LAYER

Estd : 1984

Network layer services

Packetizing:

- **Encapsulate** the upper-layer payload into a packet at the source, and **decapsulate** it at the destination.
- Source: adds a header (source/destination addresses, etc.) — payload itself is untouched unless it needs **fragmentation**
- Destination: decapsulates the packet; if fragmented, **waits for all fragments**, reassembles, then delivers upward
- **Routers along the path** cannot decapsulate or change source/destination addresses — they only **inspect addresses** to forward the packet

Routing:

- Determines the **best path** from source to destination across a network of routers/LANs/WANs
- Requires **routing protocols** to build consistent forwarding tables ahead of time

Forwarding:

- Uses the **forwarding table** (built by routing) — looks up the destination address/label to find the correct **output interface**
- Then sends the packet out through that interface

Other services:

Error control:

- Network layer designers **skipped full error control** — impractical since packets are fragmented at each router
- Instead: a **checksum on the header only** (not the whole datagram), to catch corruption in the addressing info

Flow Control:

- Regulates how much data a source sends, so the receiver isn't overwhelmed
- **Not implemented** at the network layer, because:
 - No error control here, so overload is rare
 - Upper layers already buffer and pace incoming data
 - Keeps the network layer simpler and more efficient

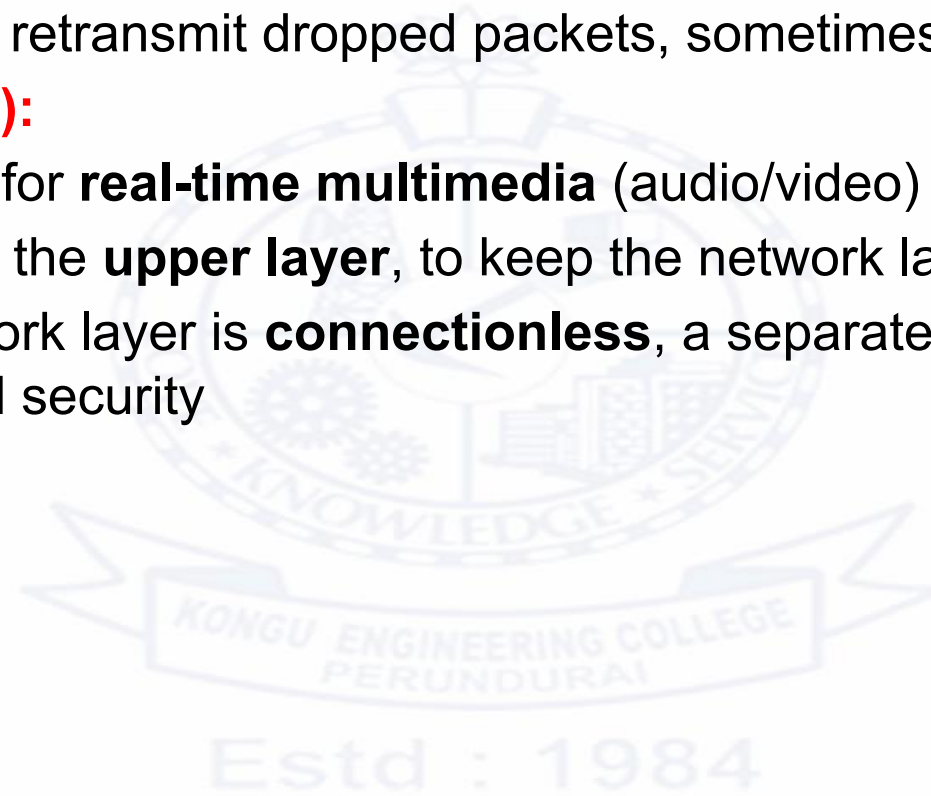
Congestion Control:

- Occurs when **too many datagrams** flood part of the network — routers may **drop packets**
- Can worsen if senders retransmit dropped packets, sometimes causing system collapse

Quality of Service (QoS):

- Increasingly important for **real-time multimedia** (audio/video)
- Mostly implemented at the **upper layer**, to keep the network layer simple

Security: since the network layer is **connectionless**, a separate virtual layer called **IPSec** adds connection-oriented security



Network layer performance

Delay:

Type	Formula	Meaning
Transmission delay	Packet length ÷ Transmission rate	Time to push all bits onto the line
Propagation delay	Distance ÷ Propagation speed	Time for a bit to travel from A to B
Processing delay	—	Time to process a packet (strip header, error-check, route)
Queuing delay	—	Time a packet waits in input/output queues at a router

****Total delay**** (n routers, source-to-destination):
$$(n + 1)(Delay_{tr} + Delay_{pg}) + n(Delay_{pr} + Delay_{qu})$$

Throughput:

- Number of bits passing a point per second
- For a path with several links in series:

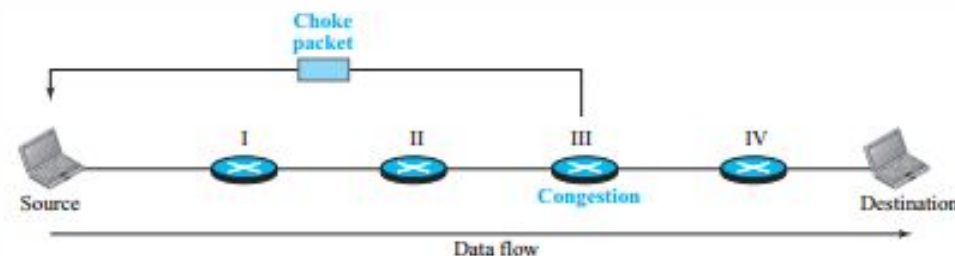
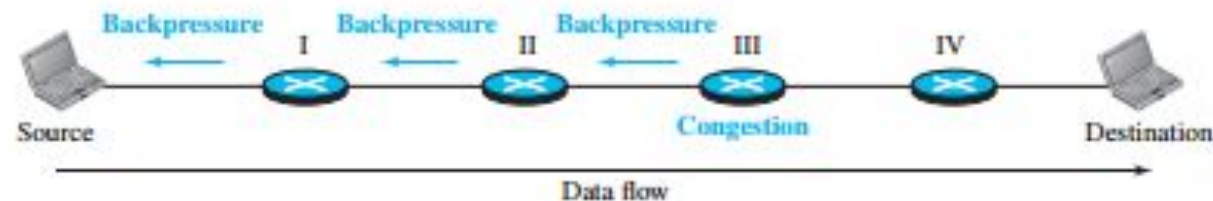
$$\text{Throughput} = \min\{TR_1, TR_2, \dots, TR_n\}$$
- The **slowest link (bottleneck)** determines overall throughput — like water flow through pipes of different diameters
- Shared links** reduce effective throughput further — if a link's capacity is split across multiple flows, only a portion is available per flow

Packet Loss:

- Occurs when a router's **input buffer overflows** and packets must be dropped
- Lost packets get **resent** — which can worsen congestion further

Congestion Control:

- Delay** stays low until capacity is reached, then rises sharply (queuing delay dominates)
- Throughput** rises with load up to capacity, then **declines** as overloaded routers start discarding packets



1. Open-Loop (Prevention) — policies applied before congestion happens:

- **Retransmission Policy** — well-designed timers prevent unnecessary retransmits
- **Window Policy** — Selective Repeat causes less congestion than Go-Back-N (fewer duplicate resends)
- **Acknowledgment Policy** — acknowledging less often reduces network load
- **Discarding Policy** — smart dropping (e.g., drop less-critical audio packets first) preserves overall quality
- **Admission Policy** — QoS mechanism; deny new virtual-circuit connections if congestion risk is high

2. Closed-Loop (Removal) — mechanisms that react after congestion occurs:

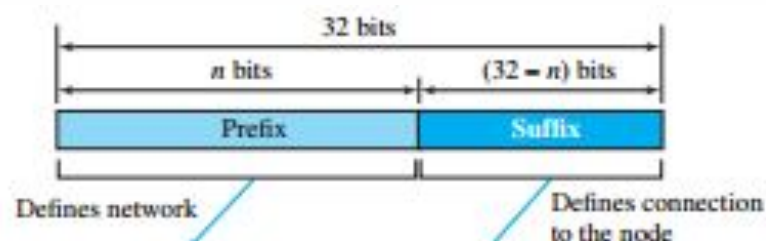
- **Backpressure** — a congested node tells its upstream neighbor to slow down; this can ripple backward node-by-node to the source. *Only works in virtual-circuit networks*, since routers there know their upstream node.
- **Choke Packet** — a congested router sends a direct warning **straight to the source** (not hop-by-hop). Used in ICMP via *source quench* messages.
- **Implicit Signaling** — no explicit message; the source *infers* congestion from symptoms (e.g., missing ACKs, delay) and slows down on its own. Used in TCP.
- **Explicit Signaling** — congestion is signaled via a dedicated bit/field carried in data packets themselves (forward or backward direction). Used in ATM networks.

IPv4 addresses

- An **IP address** = 32-bit identifier that uniquely defines a device's *connection* to the Internet
- **Address space** = total addresses possible = 2^b (b = bits used)
- IPv4 uses **32 bits** → address space = $2^{32} \approx 4.3$ billion addresses
- 3 notations:

Notation	Format	Example
Binary	32 bits, grouped in octets	10000000 00001011 00000011 00011111
Dotted-decimal	4 decimal numbers (0-255), dot-separated	128.11.3.31
Hexadecimal	8 hex digits	80 0B 03 1F

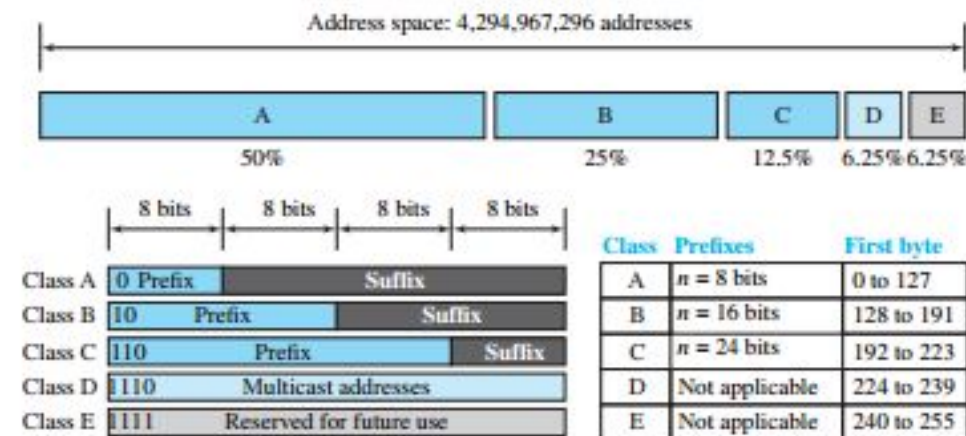
- **Hierarchy:** an IPv4 address splits into **Prefix** (defines the network, n bits) + **Suffix** (defines the node, $32-n$ bits) — like a mailing address (country→street→house).



Classful Addressing (obsolete, but foundational):

Fixed-length prefixes → 5 classes:

Class	Prefix length (n)	# Networks	First byte range
A	8	128	0-127
B	16	16,384	128-191
C	24	2,097,152	192-223
D	— (multicast)	—	224-239
E	— (reserved)	—	240-255



Problem — Address Depletion: Class A wasted huge blocks (few orgs need 16M addresses); Class C was too small for mid-size orgs → addresses ran out fast.

Attempted fixes: *Subnetting* (split a block into smaller subnets) and *Supernetting* (combine several Class C blocks) — both had limitations, leading to a better long-term approach.

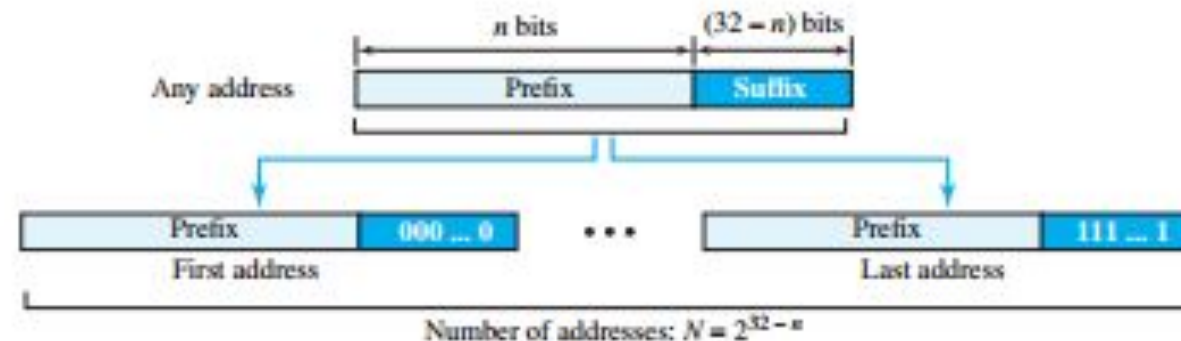
Classless Addressing:

- Removes fixed class boundaries — **variable-length prefixes** (0 to 32)
- Small prefix = large network; large prefix = small network
- Introduced with the rise of **ISPs**, who need to allocate flexible-sized blocks

Slash (CIDR-classless interdomain routing) Notation: address/n — e.g., 12.24.76.8/8, 220.8.24.255/25

Extracting info from an address, given prefix length n:

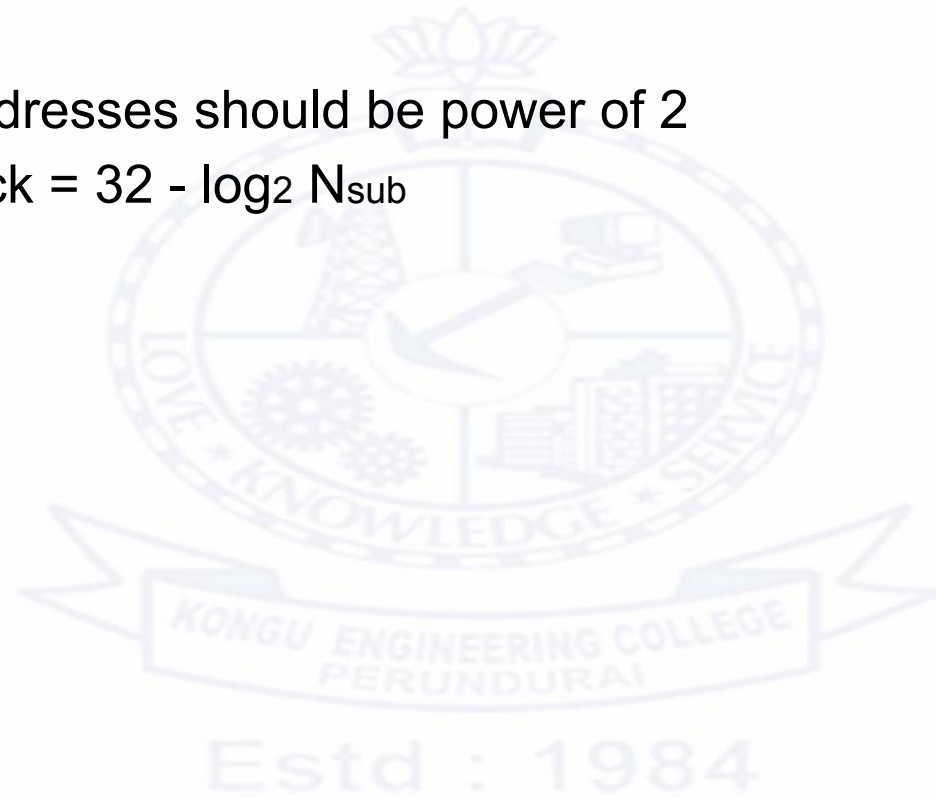
- **Number of addresses** = $2^{(32-n)}$
- **First address** = keep first n bits, set rest to 0
- **Last address** = keep first n bits, set rest to 1



Example: 167.199.170.82/27 →

- $N = 2^5 = 32$ addresses;
- Binary representation: 10100111 11000111 10101010 01010010
- First address: 10100111 11000111 10101010 010**10010**
 - = 10100111 11000111 10101010 010**00000**
 - = 167.199.170.64
- last address: 10100111 11000111 10101010 010**10010**
 - = 10100111 11000111 10101010 010**11111**
 - = 167.199.170.95

- Block allocation – global authority -> **ICANN** – Internet corporation for assigned names and numbers
- ICANN - > assign block of IP address -> ISP -> from whom we can get IP addresses
- Conditions:
 - No.of requested addresses should be power of 2
 - Prefix length of block = $32 - \log_2 N_{\text{sub}}$



- An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have 3 subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.

Step 1: Network bits = 24

Host bits = $32 - 24 = 8$ bits

Total addresses = $2^8 = 256$

First address:

00001110.00011000.01001010.00000000

14.24.74.0/24

Last address:

00001110.00011000.01001010.11111111

14.24.74.255/24

So the complete address range is: 14.24.74.0 – 14.24.74.255

We have 256 addresses available.

Requirement	Addresses needed
Subnet 1	120
Subnet 2	60
Subnet 3	10

Step 2: no.of address in larges subblock = 120 addresses -> not the power of 2

We allocate $2^7 = 128$ addresses

Prefix length = $32 - \log_2 2^7$
= $32 - 7 = 25$

1st block = 14.24.74.0/25 – 14.24.74.127/25

Step 3: Block 2 = 60 addresses -> not the power of 2

We allocate $2^6 = 64$ addresses

Prefix length = $32 - \log_2 2^6$
= $32 - 6 = 26$

1st block = 14.24.74.128/26 – 14.24.74.191/26

Step 4: Block 3 = 10 addresses -> not the power of 2

We allocate $2^4 = 16$ addresses

Prefix length = $32 - \log_2 2^4$
= $32 - 4 = 28$

1st block = 14.24.74.192/28 – 14.24.74.207/28

- If we add all the address allocated = $128 + 64 + 16 = 208$ addresses
- Total addresses = 256. So 48 IP addresses leftover.

Subnet	Required addresses	Allocated addresses	CIDR	Network Address	Usable Host Range
Subnet 1	120	128	/25	14.24.74.0	14.24.74.0 – 14.24.74.127
Subnet 2	60	64	/26	14.24.74.128	14.24.74.128 – 14.24.74.191
Subnet 3	10	16	/28	14.24.74.192	14.24.74.192 – 14.24.74.207

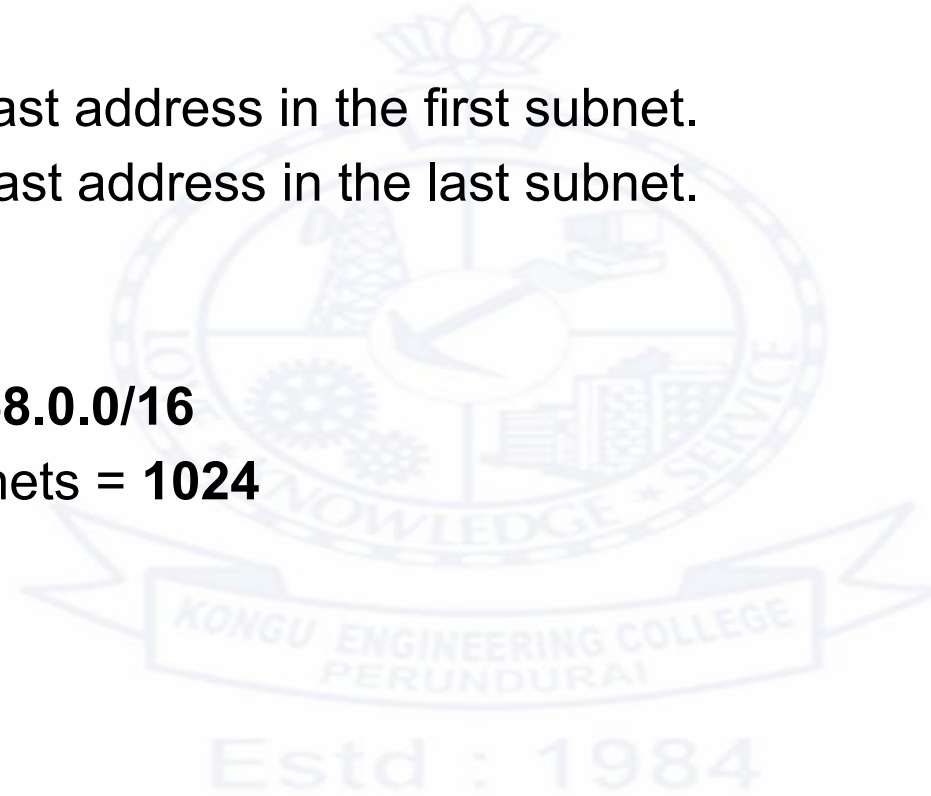
An organization is granted the block 132.58.0.0/16. The network administrator wants to create 1024 subnets.

- i) Find the number of addresses in each subnet.
- ii) Find the subnet prefix.
- iii) Find the first and the last address in the first subnet.
- iv) Find the first and the last address in the last subnet.

Given

Network address = **132.58.0.0/16**

Required number of subnets = **1024**



i) Number of addresses in each subnet

Step 1: Find how many bits are needed for 1024 subnets

We know: $1024 = 2^{10}$

So, we need to **borrow 10 bits** from the host portion to create 1024 subnets.

The original prefix is: /16

After borrowing 10 bits: $16+10=26$

Step 2: Find the number of host bits remaining

IPv4 has **32 bits**.

$32-26=6$

So, each subnet has **6 host bits**.

Therefore, number of addresses in each subnet:

$2^6=64$ addressess

ii) Find the subnet prefix

We already calculated that 10 bits are borrowed.

Original prefix:16

Borrowed bits:10

Therefore:16+10=26

iii) First and last address in the first subnet

The original network is:132.58.0.0/16

With /26, each subnet contains **64 addresses**.

So the **first subnet** starts at 132.58.0.0

The next subnet starts after 64 addresses: 132.58.0.64

First address: 132.58.0.0

Last address: 132.58.0.63

iv) First and last address in the last subnet

We have **1024 subnets**.

Each subnet contains **64 addresses**.

So total addresses used: $1024 \times 64 = 65536$

This is exactly the number of addresses in a /16 block: $2^{16} = 65536$

So the last subnet will be at the end of the /16 block.

The complete range of 132.58.0.0/16 is: 132.58.0.0 → 132.58.255.255

It contains:

132.58.255.192 → 132.58.255.255

First address: 132.58.255.192

Last address: 132.58.255.255